# RAILYATRA

Railway and Sample Flight Reservation System

Java 17 | HTML | CSS | JavaScript

Project Documentation and Demonstration Guide

**Educational prototype - not affiliated with any railway, airline or payment provider.**

Demonstration bookings, sample schedules, simulated UPI screens, and refunds only. No real money is collected and no valid travel ticket is issued.

# 1. Project Introduction

RailYatra is a responsive browser-based application developed for demonstrating the complete flow of an online railway reservation platform. The website includes sample train and flight journeys, passenger registration, route search, seat selection, simulated payment, ticket generation, and cancellation.

# 2. Problem Statement

A student learning reservation systems needs one application showing how a passenger searches a journey, selects available seats, enters passenger details, confirms a demo payment, receives a ticket reference, and cancels a booking. RailYatra combines these steps into one educational workflow.

# 3. Objectives

Provide a simple responsive website suitable for phones and laptops. Explain the relationship between frontend screens and backend processing. Validate passenger and journey inputs. Track date-specific availability and prevent duplicate confirmed-seat selection. Demonstrate payment and cancellation stages without handling real funds.

# 4. Technology Stack

| Layer | Technology / Purpose |
| --- | --- |
| Frontend | HTML, CSS, vanilla JavaScript: layout, forms, navigation, API calls |
| Backend | Java 17 HttpServer: REST-style HTTP endpoints and validation |
| Data | Java-serialized demo file: users, schedules, bookings, feedback |
| Authentication | Password hashing with PBKDF2; cookie session for demo use |
| Public hosting | Docker-based deployment on a Java-compatible web service |

# 5. Main Features and Modules

### Passenger access

Create a demo passenger account, log in, log out, and access previously created demo bookings while the data exists.

### Journey search

Choose a source, destination, travel date, train or sample flight. Browse preset schedules, departure and arrival times, fares, and available seats.

### Passenger details and seats

Enter passenger name, age and gender; select exactly the required number of seats. Red seats indicate booked seats and green indicates available seats.

### Demo checkout

Review fare, sample tax, convenience fee, optional insurance and total. Choose a simulated method such as UPI QR, UPI ID, Google Pay, PhonePe, Paytm, net banking or card.

### Ticket management

After acknowledging a demo payment, receive a generated demonstration PNR, ticket summary, and an option to download or print ticket details.

### Cancellation

Preview a simulated refund and confirm cancellation. The system records the cancellation timestamp in IST and makes seats available again.

### Other screens

Sample station board, search history, light/dark mode, language labels, feedback, and optional service-management admin tools.

## 6. Application Flow

**Register or Login -> Select Route & Date -> Find Journey -> Enter Passenger Details -> Select Seats -> Review Fare -> Confirm DEMO Payment -> View Ticket & PNR -> Cancel Ticket (optional)**

## 7. Frontend and Backend Communication

The browser uses JavaScript fetch requests to contact Java routes under /api/. The backend reads user input, validates dates and passenger details, checks bookings for seat conflicts, calculates amounts, and returns JSON responses for the frontend to display.

## 8. Storage and Data

The demonstration stores sample journeys and temporary account and ticket records in a local serialized file. This is suitable for a single-instance classroom demonstration, not a production reservation platform. Session data is held in server memory.

## 9. Sample Evaluation Checklist

| Test | Expected demonstration behavior |
|---|---|
| Registration | A new demo user can create an account. |
| Train search | Bengaluru to Chennai produces preset sample services. |
| Booking | A demo booking returns a PNR and details. |
| Duplicate seat | An already reserved seat is not offered as available. |
| Cancellation | Confirmation changes ticket status and records the time. |
| Seat release | A cancelled ticket frees seats for that travel date. |
| Website layout | The interface responds to a phone or laptop screen. |

## 10. Local Execution

Install Java JDK 17 and Apache Maven. Open the project directory containing pom.xml. Run:

```
mvn compile exec:java
```

Open http://localhost:8080 or the URL shown by the terminal. Localhost links work only on the same machine.

## 11. Public Demo Deployment

This project includes a Dockerfile and render.yaml. Upload the project files to a Git repository and deploy a Docker web service on a compatible provider. The server binds to the PORT environment variable and host address 0.0.0.0. After successful deployment, the provider supplies a public HTTPS website address.

Once published, the project document is accessible from the website at /project-document.pdf.

## 12. Limitations and Responsible Sharing

This is not an official train or flight booking portal. There is no live train location, real-time availability, real UPI payment, genuine refund, or official travel PNR. The demonstration must not request real identity documents, bank credentials, or real payment information.

On a free web-hosting plan with temporary filesystem storage, sample user accounts and bookings may disappear after a restart or redeployment. For reliable shared records, the data layer must be changed to a persistent database, and stronger security controls should be implemented.

## 13. Future Improvements

Possible future work includes a secure managed database, OTP email verification, rate limiting, improved access control, downloadable PDF tickets, properly localized content, optional maps, and approved official APIs if available.

## Conclusion

RailYatra demonstrates the complete software flow of a reservation platform using core Java and frontend web technologies. Its main purpose is to show architecture, data processing, interface design, booking validation and cancellation logic for an academic review.